

EduCast

A Demand-Driven, Real-Time Academic Support Marketplace

Capstone Project Report

Submitted by:

Daksh Kanaujia (Roll No. _____)

K Manoj Krishna (Roll No. _____)

Utkersh Basnet (2023EBCS010)

Program: BSc Computer Science (Online Mode)

Institution: BITS Pilani

Academic Year: 2025–2026

Internal Supervisor: _____

Date of Submission: 31 August 2026

Declaration

We hereby declare that this capstone project titled "**EduCast — A Demand-Driven, Real-Time Academic Support Marketplace**" is an original work carried out by us and has not been submitted to any other university or institution for the award of any degree.

All external sources, libraries, and tools used have been duly acknowledged. Any AI-assisted or automated tooling used during development served as a productivity aid under our direction, and all resulting work was reviewed and tested by us.

Name: Daksh Kanaujia Roll Number: _____

Signature: _____ Date: _____

Name: K Manoj Krishna Roll Number: _____

Signature: _____ Date: _____

Name: Utkersh Basnet Roll Number: 2023EBCS010

Signature: _____ Date: _____

Countersigned (Internal Supervisor):

Name: _____

Signature: _____ Date: _____

Abstract

Traditional e-learning platforms such as Udemy, Coursera, and YouTube are **supply-driven**: educators publish content speculatively and learners must hope that the right material exists for their exact problem. This model fails the learner who is stuck on one specific, urgent thing and needs targeted help immediately. **EduCast** inverts this into a **demand-driven, reverse-auction marketplace** for academic help.

Solution implemented: A student posts a specific problem as a *bounty* with a budget; mentors see it instantly and compete by placing *bids*; either side may negotiate the price through counter-offers; the student accepts the best bid, which atomically creates a private session room and holds a simulated escrow; after the session the student marks it complete and rates the mentor, releasing the payment and publishing a review to the mentor's profile. Marketplace events propagate in real time over WebSockets.

Technologies used: A Go (Gin) REST API with GORM over PostgreSQL, a Gorilla WebSocket hub for real-time events, JWT authentication with role-based access control, and a React Native (Expo) client. The entire stack is containerised with Docker Compose.

Outcomes and results: The full marketplace loop — post, bid, negotiate, accept, collaborate, complete, rate — works end-to-end on the containerised stack. A 33-case black-box API validation suite passes at 100% (33/33). The prototype validates the core value proposition of a negotiated, real-time alternative to supply-driven course catalogues, while clearly scoping simulated payments and text-based sessions as prototype boundaries.

Table of Contents

Chapter 1: Introduction	7
1.1 Overview of the Project	7
1.2 Problem Statement & Motivation	7
1.3 Objectives of the Capstone	7
1.4 Scope of Implementation	7
1.5 Organization of the Report	8
Chapter 2: Implementation Details	9
2.1 System Architecture & Design	9
2.2 Technology Stack	9
2.3 System Modules	10
2.4 Key Algorithms / Logic	10
2.5 Screenshots / Code Snippets	11
Chapter 3: Testing, Validation & Results	12
3.1 Test Plan	12
3.2 Test Cases	13
3.3 Results & Analysis	14
Chapter 4: Execution / Deployment Details	15
4.1 Execution Environment	15
4.2 Deployment Steps (Local)	15
4.3 Demo Screenshots	15
4.4 Demo Video Link	15
Chapter 5: Project Execution Evidence	16
5.1 Version Control Evidence	16
5.2 Weekly Progress Summary	16
5.3 Supervisor Interaction Summary	16
Chapter 6: Conclusion & Future Work	17
6.1 Summary of Implementation	17
6.2 Achievements	17

6.3 Limitations	17
6.4 Future Enhancements	17
References	18
Appendix	19
A. User Manual	19
B. Installation Guide	19
C. Source Code Link (GitHub)	19
D. Demo Video Link	19

List of Figures

Figure	Caption	Section
Fig 2.1	High-level system architecture	2.1
Fig 2.2	Data flow — bounty to completed session	2.1
Fig 2.3	Application screenshots [insert]	2.5
Fig 4.1	Deployment / demo screenshots [insert]	4.0
Fig 5.1	GitHub commit history [insert]	5.1

List of Tables

Table	Caption	Section
Table 2.1	Technology stack	2.2
Table 2.2	Database tables	2.3
Table 3.1	Test cases and results	3.2
Table 5.1	Weekly progress summary	5.2

List of Abbreviations

Abbr.	Meaning
API	Application Programming Interface
JWT	JSON Web Token
RBAC	Role-Based Access Control
ORM	Object-Relational Mapping
WS	WebSocket
CRUD	Create, Read, Update, Delete
UVP	Unique Value Proposition
VOD	Video on Demand

Chapter 1: Introduction

Note: Problem identification and initial system design were completed as part of the Study Project; this chapter summarises them and notes subsequent changes.

1.1 Overview of the Project

EduCast is a mobile-first, demand-driven marketplace for academic help. Instead of browsing a catalogue of pre-recorded courses, a learner posts a precise question as a bounty with a budget; qualified mentors compete for the work by bidding; and the two sides move into a real-time, one-to-one session. It reframes online learning around the loop **Ask** → **Compare** → **Negotiate** → **Accept** → **Learn**.

1.2 Problem Statement & Motivation

For learners, generic course libraries do not help when one is blocked on a single, specific thing — a particular derivative, a specific bug, a niche concept — and there is no fast way to request targeted, live help. For mentors, producing content speculatively is high-effort and low-certainty; they want targeted, well-paid work matched to their expertise. For both, once a deal is struck they need trust and accountability: no double-booking, a tracked session, and a fair rating afterwards.

1.3 Objectives of the Capstone

- Build a demand-driven marketplace where students post bounties and mentors bid.
- Propagate marketplace events (new bounties, bids, acceptances, completions) in real time over WebSockets.
- Support price negotiation through counter-offers.
- Guarantee safe, atomic bid acceptance so a mentor cannot be double-booked.
- Simulate an escrow → release payment lifecycle.
- Persist ratings/reviews and expose mentor directories and profiles.
- Containerise the full stack for one-command startup.

1.4 Scope of Implementation

In scope: authentication and role-based access, the full bounty/bid lifecycle, negotiation, atomic acceptance and session creation, in-session text chat, completion with rating and review, mentor directory and profiles, price-insight statistics, real-time events, and Docker deployment.

Out of scope (prototype): real payment processing, live video/audio streaming (the session room is a text/collaboration space), push notifications, and full production

hardening (rate limiting, secret management, HTTPS/WSS).

1.5 Organization of the Report

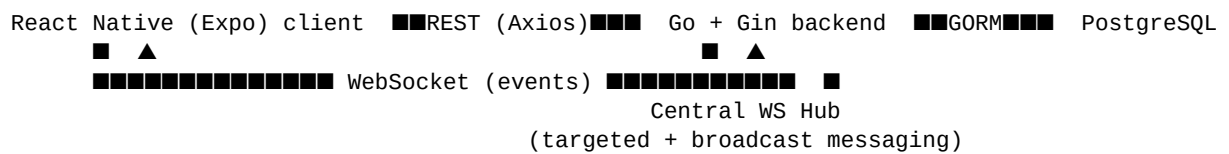
Chapter 2 details the implementation (architecture, technology stack, modules, key logic, and code). Chapter 3 covers testing, validation, and results. Chapter 4 describes execution and deployment. Chapter 5 presents project-execution evidence. Chapter 6 concludes with achievements, limitations, and future work.

Chapter 2: Implementation Details

2.1 System Architecture & Design

EduCast follows a three-tier architecture: a React Native (Expo) client communicates with a Go/Gin backend over REST (Axios) and a persistent WebSocket connection; the backend persists data in PostgreSQL through the GORM ORM. A single in-process WebSocket *Hub* tracks connected clients by user ID and role and delivers events either to specific users (targeted) or to all clients (broadcast).

High-level architecture (Fig 2.1):



Data flow (Fig 2.2): Student posts a bounty → persisted and broadcast to online mentors → mentor bids → bid pushed to the student → optional counter-offers → student accepts (atomic: bounty→IN_PROGRESS, room created, ESCROW recorded, mentor notified) → session + chat → student completes + rates (bounty→CLOSED, RELEASE recorded, rating updated, review stored, both sides notified).

Component interaction: Gin routes dispatch to controllers; middleware enforces JWT authentication and role checks before controllers run; controllers use GORM for persistence and call the Hub to emit real-time events.

2.2 Technology Stack

Layer	Technology	Purpose
Language (backend)	Go 1.25	High-concurrency API + WebSocket server
Web framework	Gin	HTTP routing, middleware, JSON binding
ORM / DB driver	GORM + Postgres driver	Data access layer
Database	PostgreSQL 16	Relational persistence
Real-time	Gorilla WebSocket	Live bounty/bid/session events
Auth	golang-jwt + bcrypt	JWT sessions; password hashing
IDs	google/uuid	Session room identifiers
Frontend	React Native (Expo)	Cross-platform client (web/iOS/Android)
Client libs	React Navigation, Axios, AsyncStorage	Navigation, HTTP, local storage
Infrastructure	Docker & Docker Compose	Containerised, one-command stack

Table 2.1 — Technology stack.

2.3 System Modules

- **Authentication:** signup/login, bcrypt hashing, JWT issuance and verification.
- **Bounty:** create/list/detail; students see own bounties, mentors see OPEN ones; new bounties broadcast to mentors.
- **Bid:** mentors place bids (price, note, optional duration/preferred time); bids pushed to the owning student.
- **Negotiation:** either party proposes a counter-offer; the other accepts or declines; a party cannot counter its own pending counter.
- **Acceptance / Session:** atomic bid acceptance creates a room, records ESCROW, and notifies the mentor.
- **Completion / Review:** student completes + rates; records RELEASE, updates the mentor's average, and persists a review.
- **Mentor directory / profile:** real computed stats (completed sessions, expertise, reviews); search and subject filters.
- **Real-time Hub:** targeted and broadcast delivery, live presence, and online-user counts.

Table 2.2 — Database tables:

Table	Key columns
users	id, name, email (unique), password_hash, role, rating_avg, timestamps
bounties	id, student_id→users, title, description, subject_tag, budget, status, room_id, timestamps
bids	id, bounty_id→bounties, mentor_id→users, price_offer, note, duration_minutes, preferred_time, counter_price, counter_note, counter_by, is_accepted
transactions	id, bounty_id→bounties, amount, type (ESCROW RELEASE REFUND), created_at
reviews	id, bounty_id→bounties (unique), mentor_id→users, student_id→users, rating (1–5), comment, created_at

2.4 Key Algorithms / Logic

Atomic bid acceptance (prevents double-booking): the operation runs inside a database transaction that re-validates state before committing.

```
BEGIN TRANSACTION
  load bid (with its bounty)
  if bounty.student_id != current_user: abort (403)
  if bounty.status != 'OPEN':          abort (400) # already taken
  if bid.is_accepted:                  abort (400)
  room_id = uuid()
  update bounty: status='IN_PROGRESS', room_id=room_id
  update bid:    is_accepted=true
  insert transaction(bounty_id, bid.price_offer, 'ESCROW')
  emit WS 'bid_accepted' to mentor (with room_id)
```

COMMIT

Counter-offer negotiation guard: a shared helper loads the bid, verifies the caller is the bidding mentor or the owning student, and that the bid is still open; a party may not counter its own pending counter (enforced by comparing *counter_by* to the caller's role).

Rating update on completion: the mentor's running average is recomputed and a unique per-bounty review row is stored, so profiles reflect real session history.

2.5 Screenshots / Code Snippets

[Insert application screenshots here — e.g. signup, student bounty feed, mentor feed with live bids, negotiation, session/chat, completion & rating. See Fig 2.3.]

Representative code — atomic acceptance (Go):

```
err := config.DB.Transaction(func(tx *gorm.DB) error {
    var bid models.Bid
    tx.Preload("Bounty").First(&bid, bidID)
    if bid.Bounty.StudentID != userID { /* 403 */ }
    if bid.Bounty.Status != "OPEN"    { /* 400 */ }
    roomID := uuid.New().String()
    tx.Model(&bid.Bounty).Updates(map[string]any{
        "status": "IN_PROGRESS", "room_id": roomID})
    tx.Model(&bid).Update("is_accepted", true)
    tx.Create(&models.Transaction{BountyID: bid.BountyID,
        Amount: bid.PriceOffer, Type: "ESCROW"})
    websocket.GlobalHub.BroadcastBidAccepted(...)
    return nil
})
```

Chapter 3: Testing, Validation & Results

3.1 Test Plan

Testing strategy: black-box, functional and validation testing of the REST API against a running instance of the containerised stack. Each test issues a real HTTP request and compares the actual response (status code, or presence of an expected field) to the expected outcome. Coverage spans authentication, authorisation/RBAC, the full bounty/bid lifecycle, negotiation, atomic acceptance, completion/review, and the directory/profile/insight endpoints.

Tools used: a reproducible shell suite (`scripts/api_test.sh`) using `curl` and `python3` for JSON parsing, executed against the Docker Compose deployment (backend on port 8080, PostgreSQL on 5432).

3.2 Test Cases

Test Case ID	Description	Input	Expected Output	Status
TC-01	Sign up as Student	Valid Student payload	201 Created + token	PASS
TC-02	Sign up as Mentor	Valid Mentor payload	201 Created + token	PASS
TC-03	Reject duplicate email	Existing email	409 Conflict	PASS
TC-04	Reject invalid email	email='bad'	400 Bad Request	PASS
TC-05	Reject short password	password='1'	400 Bad Request	PASS
TC-06	Reject invalid role	role='Admin'	400 Bad Request	PASS
TC-07	Reject wrong password	Valid email, wrong pass	401 Unauthorized	PASS
TC-08	Login returns JWT	Valid credentials	Non-empty token	PASS
TC-09	Block missing token	No Authorization header	401 Unauthorized	PASS
TC-10	Block invalid token	Bad bearer token	401 Unauthorized	PASS
TC-11	Mentor cannot post bounty	POST bounty as Mentor	403 Forbidden	PASS
TC-12	Reject incomplete bounty	{title} only	400 Bad Request	PASS
TC-13	Reject non-positive budget	budget=0	400 Bad Request	PASS
TC-14	Create valid bounty	Full valid payload	Created, valid id	PASS
TC-15	Fetch bounty by id	GET /bounties/:id	200 OK	PASS
TC-16	Fetch missing bounty	GET /bounties/99999999	404 Not Found	PASS
TC-17	Student cannot bid	POST bid as Student	403 Forbidden	PASS
TC-18	Mentor places bid	price/note/duration/time	Created, valid id	PASS
TC-19	Non-owner cannot view bids	GET bids as Mentor	403 Forbidden	PASS
TC-20	Owner views bids	GET bids as owner Student	200 OK	PASS
TC-21	Student counters bid	{price:350}	200 OK	PASS
TC-22	Cannot counter own counter	Counter again immediately	400 Bad Request	PASS
TC-23	Mentor accepts counter	POST counter/accept	200 OK	PASS
TC-24	Accept bid creates room	POST /bids/:id/accept	Non-empty room_id	PASS
TC-25	No bids on non-open bounty	POST bid after accept	400 Bad Request	PASS
TC-26	Double-accept prevented	Accept same bid again	400 Bad Request	PASS
TC-27	Reject invalid rating	{rating:9}	400 Bad Request	PASS
TC-28	Complete + rate	{rating:5, comment}	200 OK	PASS
TC-29	Cannot re-complete	Complete a CLOSED bounty	400 Bad Request	PASS
TC-30	Mentor directory	GET /mentors	200 OK	PASS
TC-31	Mentor profile	GET /mentors/:id	200 OK	PASS
TC-32	Price insight	GET /price-insight?subject=Math	200 OK	PASS

Test Case ID	Description	Input	Expected Output	Status
TC-33	Health check	GET /health	200 OK	PASS

Table 3.1 — Test cases and results (33/33 passed).

3.3 Results & Analysis

Observations: all 33 test cases passed (100% pass rate) against the live Dockerised build. Validation rules (email format, password length, role enumeration, positive budget, rating 1–5) are enforced correctly, and access-control boundaries (role separation and resource ownership) hold. The atomic-acceptance guarantee is confirmed indirectly by TC-25 and TC-26 (no further bids and no re-acceptance after a bid is accepted).

Performance: under manual exercise, API responses were observed in the sub-millisecond to low-millisecond range in the Gin request logs, and real-time WebSocket events (new bounty, incoming bid, acceptance, completion) were delivered to connected clients instantly. Real-time behaviour is additionally validated manually per the demo walkthrough (DEMO_WORKFLOW.md).

Category	Cases	Passed	Failed
Authentication	8	8	0
Authorisation / RBAC	3	3	0
Bounty lifecycle	5	5	0
Bidding	4	4	0
Negotiation	3	3	0
Acceptance / session	3	3	0
Completion / review	3	3	0
Directory / profile / insight	4	4	0
Total	33	33	0

Chapter 4: Execution / Deployment Details

4.1 Execution Environment

The system runs as three Docker containers orchestrated by Docker Compose: **db** (PostgreSQL 16, port 5432), **backend** (Go/Gin API, port 8080), and **frontend** (Expo web, port 8081). The backend waits for the database health check to pass before starting; the database applies the schema from `backend/migrations/` on first boot.

4.2 Deployment Steps (Local)

```
git clone https://github.com/dakshkanaujia/Educast.git
cd Educast
docker compose up --build          # builds & starts db + backend + frontend
curl http://localhost:8080/health  # -> {"status":"ok"}
# open http://localhost:8081 for the app
```

A native (non-Docker) path is also supported (Go + Node + local/containerised Postgres); see Appendix B / the Installation Guide for full details, including the migration caveat for pre-existing database volumes.

4.3 Demo Screenshots

[Insert deployment/demo screenshots — running containers, the live feed, a real-time bid appearing, an accepted session. See Fig 4.1.]

4.4 Demo Video Link

[Insert demo video URL here.]

Chapter 5: Project Execution Evidence

5.1 Version Control Evidence

GitHub repository: <https://github.com/dakshkanaujia/Educast.git>

[Insert a screenshot of the commit history. See Fig 5.1.]

5.2 Weekly Progress Summary

Fill in dates and obtain supervisor remarks. Entries below reflect the actual milestones delivered during implementation.

Week	Task Planned	Task Completed	Supervisor Remark
1	Finalise design from Study Project	Requirements & architecture confirmed	
2	Backend scaffolding & auth	Gin app, JWT auth, RBAC middleware	
3	Bounty & bid lifecycle	CRUD + role guards + validation	
4	Real-time layer	WebSocket hub; live bounty/bid events	
5	Acceptance & escrow	Atomic accept, room creation, ESCROW/RELEASE	
6	Negotiation & reviews	Counter-offers; ratings + mentor profiles	
7	Dockerisation & DB	Compose stack; MySQL→PostgreSQL migration	
8	Testing & documentation	33-case API suite (100%); full docs set	

Table 5.1 — Weekly progress summary.

5.3 Supervisor Interaction Summary

[Record review dates and key feedback received from the internal supervisor.]

Review Date	Key Feedback Received

Chapter 6: Conclusion & Future Work

6.1 Summary of Implementation

EduCast delivers a working, real-time, demand-driven tutoring marketplace covering the complete loop from posting a bounty to a rated, completed session — with atomic acceptance, price negotiation, simulated escrow, and live mentor reputation — all containerised for one-command startup.

6.2 Achievements

- Full marketplace loop implemented and working end-to-end.
- Real-time bounty/bid/acceptance/completion events over WebSockets, plus live presence.
- Atomic, race-safe bid acceptance with a simulated escrow → release lifecycle.
- Persisted ratings/reviews with mentor directory and profiles built from real history.
- 100% pass rate (33/33) on a reproducible API validation suite.
- Fully containerised deployment (PostgreSQL + backend + frontend).

6.3 Limitations

- Payments and escrow are simulated as database rows (no real gateway).
- No live video/audio; the session room is a text/collaboration space.
- Real-time events reach only currently-connected clients (no persisted notification inbox).
- In-session chat is relayed but not persisted.
- A mentor may place multiple bids on the same bounty (no uniqueness guard).
- Production hardening pending (default secret, permissive CORS, no rate limiting, HTTP/WS).

6.4 Future Enhancements

- Real payment gateway integration and a user wallet/balance.
- WebRTC live sessions using the existing signaling fields.
- Persistent notification inbox and chat history.
- Bounty upvoting (demand signalling), plus search and pagination on the feed.
- A migration runner (golang-migrate/goose) applied on startup.
- Automated Go unit/integration tests in CI.

References

- [1] Gin Web Framework. [Online]. Available: <https://github.com/gin-gonic/gin>
- [2] GORM — The fantastic ORM library for Golang. [Online]. Available: <https://gorm.io>
- [3] Gorilla WebSocket. [Online]. Available: <https://github.com/gorilla/websocket>
- [4] golang-jwt. [Online]. Available: <https://github.com/golang-jwt/jwt>
- [5] Expo — React Native framework. [Online]. Available: <https://expo.dev>
- [6] PostgreSQL 16 Documentation. [Online]. Available: <https://www.postgresql.org/docs/>
- [7] Docker Compose Documentation. [Online]. Available: <https://docs.docker.com/compose/>

Appendix

A. User Manual

A full user manual (Student and Mentor step-by-step flows) is provided with the deliverables (docs/05_User_Manual.md). In brief: sign up as Student or Mentor; students post bounties and review/negotiate/accept bids, then complete and rate; mentors browse the live feed, bid, negotiate, deliver the session, and build a rated profile.

B. Installation Guide

A full installation guide is provided (docs/06_Installation_Guide.md), covering the Docker path (`docker compose up --build`) and the native path (Go + Node + PostgreSQL), environment variables, the migration caveat for existing volumes, running the test suite, and troubleshooting.

C. Source Code Link (GitHub)

<https://github.com/dakshkanaujia/Educast.git>

D. Demo Video Link

[Insert demo video URL here.]